TECHADEMY

FORGE — FDE ACADEMY · WEEK-2 INTEGRATION · COACHING #2

Ship It — Safely

Container Apps · CI/CD · LLMOps · 02:00 Rollback

Deploy the Python agent on Azure Container Apps · wire a GitHub Actions pipeline with blue/green and an eval gate that blocks bad deploys · rehearse the 02:00 rollback runbook that stands between you and a SEV-1

Container Apps | ACR | blue/green | eval gate | rollback runbook

Trainer · Lakshminarayanan

SESSION MAP

What we cover today

Deploy on Container Apps

When CA wins vs AKS — AKS as read-ahead, not built live

1

Dockerise · ACR · auto-scale

Package the Python agent, push to ACR, set KEDA scale rules

2

CI/CD with GitHub Actions

Blue/green deploy + an eval gate that blocks bad deploys

3

LLMOps

Rollback patterns · prompt versioning · model-card automation

4

02:00 rollback · BFSI go-live

A bad prompt broke prod — the runbook is all that stops a SEV-1

5

Coaching #2 + capstone

Week 1–2 retro · confidence map · Week-3 readiness · capstone

6

OUTCOMES

By the end, you will be able to…

Pick the platform

Justify Container Apps vs AKS for an agent — and know when to graduate to AKS.

Containerise & scale

Dockerise the Python agent, push to ACR, and set scale-to-zero rules.

Automate the release

Wire a GitHub Actions pipeline with blue/green and an eval gate.

Operate the LLM

Version prompts, auto-generate model cards, and roll back in one command.

Survive 02:00

Execute a rehearsed rollback runbook before a bad deploy becomes a SEV-1.

Ship the capstone

Deliver an Azure-deployed agent with eval gates, telemetry and a rollback play.

TECHADEMY

01

DEPLOY ON CONTAINER APPS

When CA wins vs AKS

Container Apps wins live; AKS is read-ahead

Choose Container Apps when…

- You want serverless containers — no cluster, no kubectl, no node pools.
- Traffic is variable: scale-to-zero means idle costs nothing.
- You want image ⟶ HTTPS endpoint in under 10 minutes.
- Built-in KEDA autoscaling, revisions and traffic-splitting — no setup.
- The team's energy should go to the agent, not to operating Kubernetes.

Read AKS ahead for when…

- Your runbook needs kubectl or the raw Kubernetes API.
- You need custom CNI / network policies, DaemonSets or node agents.
- Sustained high utilisation (>~40%/month) makes reserved nodes cheaper.
- You need CRDs, custom operators, or multi-cloud portability.
- AKS Automatic narrows the ops gap — but idle nodes still bill.

Today we deploy live on Container Apps. AKS is your read-ahead: the platform you graduate to when a real constraint — not habit — demands it.

SIDE BY SIDE

Container Apps vs AKS at a glance

	Azure Container Apps	AKS
Abstraction	Serverless — K8s hidden (KEDA + Envoy + Dapr)	Full Kubernetes API + control plane
Scaling	KEDA rules built-in; genuine scale-to-zero	HPA + Cluster Autoscaler + KEDA — you wire it
kubectl / K8s API	Not exposed	Full access
Ops burden	Minimal — no nodes to patch	Cluster upgrades, node pools, SRE playbooks
Blue/green	Native: revisions + labels + traffic weights	Manual: Deployments + Services / service mesh
Cost fit	Variable / bursty; pay-per-second	Sustained high utilisation; reserved nodes
Best for today	Ship the agent fast, safely	Read-ahead — graduate when constrained

TECHADEMY

02

DOCKERISE · ACR · AUTO-SCALE

Package the Python agent

Dockerising the Python agent

What good looks like

- Slim base image (python:3.12-slim) — faster pulls, smaller attack surface.
- Multi-stage build: install deps in one stage, copy only what runs into the final image.
- Non-root user; pin dependency versions for reproducible builds.
- Bind to 0.0.0.0:$PORT — Container Apps injects the port and health-probes it.
- One process per container; keep the agent stateless — state lives in Cosmos DB.

Dockerfile

```
# Dockerfile — FastAPI agent
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
RUN useradd -m appuser
COPY --from=build /usr/local/lib/... .
COPY . .
USER appuser
EXPOSE 8000
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0",
     "--port", "8000"]
```

Push to Azure Container Registry

Two ways to build & push

- Local: docker build → tag with the registry → docker push.
- Cloud: az acr build runs the build on ACR — no local Docker needed.
- Tag every image with the commit SHA — never rely on :latest in prod.
- Let Container Apps pull with a Managed Identity granted the AcrPull role — no passwords.

acr_push.sh

```
# tag with the immutable commit SHA
export TAG=$(git rev-parse --short HEAD)
export ACR=bankagentacr

# option A — build in the cloud (no Docker)
az acr build \
  --registry $ACR \
  --image agent:$TAG .

# option B — local build + push
az acr login --name $ACR
docker build -t $ACR.azurecr.io/agent:$TAG .
docker push $ACR.azurecr.io/agent:$TAG

# app pulls via Managed Identity (AcrPull)
# -> no registry password in the app
```

Auto-scaling rules (KEDA)

How ACA scaling works

- Set minReplicas / maxReplicas and one or more scale rules.
- minReplicas: 0 = scale-to-zero — pay nothing when idle (cold start ~5–30s).
- Scale on HTTP concurrency, queue depth, CPU, or any of 60+ KEDA scalers.
- For latency-sensitive agents, set minReplicas: 1 to keep one warm replica.

scale.yaml

```
# containerapp scale section
properties:
  template:
    scale:
      minReplicas: 1      # 1 warm replica
      maxReplicas: 30
      rules:
      - name: http-rule
        http:
          metadata:
            concurrentRequests: '50'
      - name: queue-rule     # burst work
        custom:
          type: azure-servicebus
          metadata:
            queueName: agent-tasks
            messageCount: '20'
```

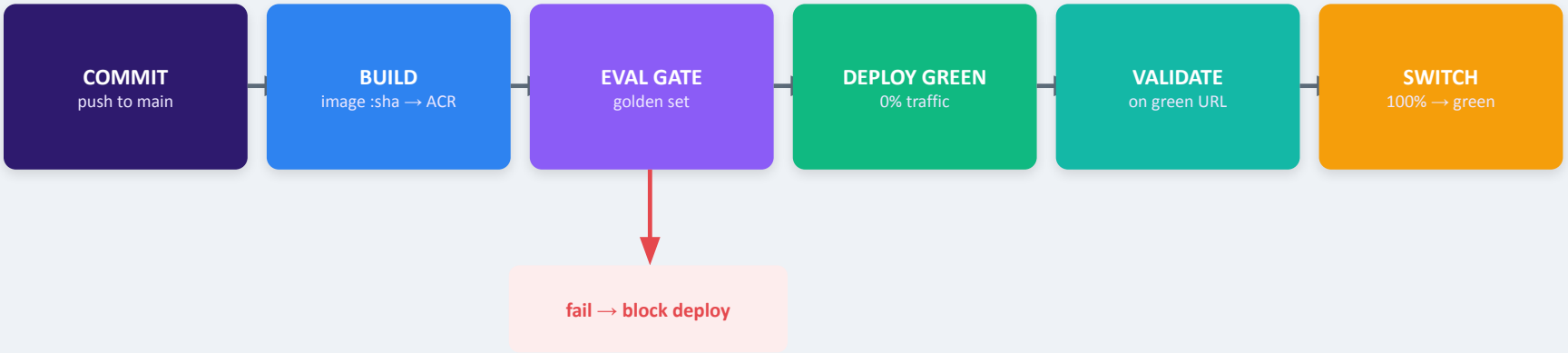
TECHADEMY

03

CI/CD WITH GITHUB ACTIONS

Blue/green + an eval gate

From commit to production, safely



Two safety nets, one pipeline

The eval gate stops a bad change before it ever reaches users. Blue/green means the switch is a traffic flip, not a rebuild — so if something slips through, rollback is instant. Keep the old (blue) revision until green is proven.

GitHub Actions: build → push → deploy

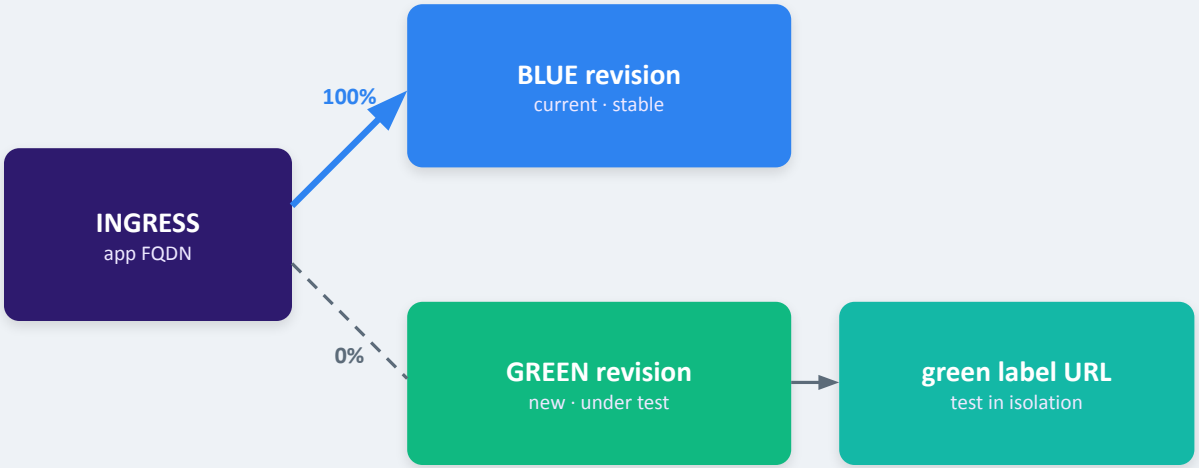
Key choices

- Authenticate with OIDC (federated) — no long-lived secret in GitHub.
- id-token: write permission lets the runner mint a short-lived Azure token.
- Build once, tag with github.sha, push to ACR.
- Deploy with the container-apps action or az containerapp update.
- Gate sits between build and deploy — a failed job blocks the release.

deploy.yml

```
name: deploy-agent
on: { push: { branches: [main] } }
permissions:
  id-token: write    # OIDC, no secrets
  contents: read
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
      with:
        client-id: ${{ vars.AZ_CLIENT_ID }}
        tenant-id: ${{ vars.AZ_TENANT_ID }}
        subscription-id: ${{ vars.AZ_SUB }}
    - run: az acr build -r $ACR \
        -t agent:${{ github.sha }} .
```

Blue/green with revisions & labels



promote.sh

```
# promote
az containerapp \
 ingress traffic set \
 --label-weight \
   blue=0 green=100

# rollback
  blue=100 green=0
```

Each deploy is an immutable revision

Set activeRevisionsMode: multiple. Label the old revision blue, the new one green. Validate green on its own URL (app---green.<env>...) at 0% traffic, then shift weight to 100%. The standby revision costs nothing on Consumption — it scales to zero.

An eval gate that blocks bad deploys

How the gate works

- Any prompt, model or retrieval change triggers an eval run in CI.
- Score the change against a versioned golden set (correctness · faithfulness · relevance).
- Compare to the current production baseline — not an absolute number.
- If any metric drops >2% below baseline, fail the job → the deploy is blocked.
- Publish results.json as an artifact and post a summary to the PR.

eval_gate.yml

```
eval-gate:
  needs: build
  runs-on: ubuntu-latest
  steps:
  - run: python -m eval.harness \
      --golden data/golden.jsonl \
      --baseline prod-baseline.json \
      --out results.json
  - run: |
      python - <<'PY'
      import json,sys
      r=json.load(open('results.json'))
      if r['regression'] > 0.02:
          sys.exit(1)    # BLOCK deploy
      PY
  - uses: actions/upload-artifact@v4
    with: { path: results.json }
```

TECHADEMY

04

LLMOPS

Rollback · prompt versioning · model cards

Rollback patterns

Traffic flip

Blue/green: shift 100% back to the last-good revision. Instant, no rebuild — the fastest lever you have.

Prompt rollback

Prompts are versioned artifacts. Re-point the registry to the previous version — a single command, seconds.

Config / flag flip

A feature flag or routing-config change propagates in <60s — no deploy needed to disable a bad path.

Post-mortem → golden set

Every rollback ends by adding the failure case to the golden set, so the gate catches it next time.

Target: detect a regression within ~15 minutes and revert in under a minute. Rollback is a rehearsed routine, not an improvisation.

Prompt versioning

Why version prompts

- A prompt is a deployment artifact — a wording tweak changes production behaviour.
- Store prompts in a registry / versioned file — the single source of truth, code-reviewed.
- Every version has an id, an author, and the eval scores it passed.
- Prompt-diff review is its own skill — small wording changes can shift tone and accuracy.
- Rollback = re-point to the previous version id. No redeploy of the app.

registry.yaml

```
# prompts/registry.yaml (source of truth)
active: kyc-triage@v7
versions:
  kyc-triage@v7:
    author: lakshmi
    commit: c6f1515
    eval: { correctness: 0.94,
            faithfulness: 0.91,
            kappa_vs_human: 0.79 }
    prompt: |
      You are a KYC triage assistant...
  kyc-triage@v6:      # last known good
    eval: { correctness: 0.93, ... }

# rollback: set active -> kyc-triage@v6
```

Model-card automation

What a model card is

- A structured record: intended use, data, eval results, limitations, ethics.
- The bridge between who built the model and who deploys it — auditors read it.
- Standard since Mitchell et al. (2019); used by Hugging Face, Google & the EU AI Act.
- Automate it: generate the card in CI from eval results + version metadata.
- For BFSI it doubles as EU AI Act high-risk technical documentation.

model_card.yaml

```yaml
# generated in CI on every release
---
model: claude-sonnet-4-6
prompt_version: kyc-triage@v7
commit: c6f1515
intended_use: KYC triage (BFSI)
out_of_scope: [credit decisions, legal advice]
eval:
  golden_set: kyc-500
  correctness: 0.94
  faithfulness: 0.91
  kappa_vs_human: 0.79
limitations:
  - EN only; escalate other languages
  - human sign-off on high-risk cases
owner: risk-platform@bank
generated: 2026-07-10T02:14Z
```

TECHADEMY

05

PRODUCTION ROLLBACK AT 02:00

Your runbook vs a SEV-1

SCENARIO

02:00 — a bad prompt change broke prod

01:58	A late prompt tweak merges and auto-deploys. It skipped review; the eval gate was disabled 'just this once'.
02:00	PagerDuty fires: faithfulness alarm + a spike in customer-complaint intents. The agent is confidently wrong on balances.
02:04	You open telemetry: p99 is fine, errors are low — but the eval monitor shows a sharp quality drop since 01:58.
02:06	You don't debug the prompt at 2am. You execute the runbook: flip traffic to the last-good revision.
02:07	Traffic is 100% blue again. Quality recovers. You file the SEV, and the bad prompt becomes a golden-set case.

The lesson isn't 'debug faster'. It's that a rehearsed rollback beats a heroic 2am fix every time.

02:00 rollback — the only thing that matters

Runbook steps (pinned, tested)

1. Confirm: is quality actually down? Check the eval / faithfulness monitor.
2. Declare: open a SEV, post in the incident channel, assign an IC.
3. Roll back: shift 100% traffic to the last-good (blue) revision.
4. Verify: quality recovers on the production FQDN within minutes.
5. Freeze: block further deploys until the cause is understood.
6. Post-mortem: add the failing input to the golden set; re-enable the gate.

The one command

```
az containerapp ingress traffic set \
  -n bank-agent -g rg-prod \
  --label-weight blue=100 green=0
```

rollback.sh

```
# pin the last-good revision up front
LAST_GOOD=bank-agent--fb699ef

# rollback is a label flip — no rebuild,
# no image pull, standby already warm
# => production restored in seconds

# THEN: freeze deploys, start post-mortem
```

A bank agent goes live safely

The go-live setup

- A KYC-triage agent on Container Apps, containerised and pushed to ACR.
- GitHub Actions CI/CD: build → eval gate → blue/green → traffic switch.
- Eval gate on a 500-case golden set; deploy blocked if faithfulness drops >2%.
- Two independent reviewers sign off high-risk changes (regulated tier).
- Every release ships a generated model card = EU AI Act technical documentation.
- A pinned 02:00 rollback runbook: one traffic-flip to the last-good revision.

Why it holds under audit

- Traceable: every deploy ties to a commit, prompt version and eval score.
- Reversible: rollback rehearsed, sub-minute, no rebuild.
- Documented: dated model card per release for the regulator.

Severity ladder (agree before go-live)

SEV-1	Wrong financial info to customers → roll back now
SEV-2	Quality dip, contained → roll back, fix in hours
SEV-3	Minor / cosmetic → fix forward in next release

TECHADEMY

06

COACHING #2

Week 1–2 retro · readiness

Week 1–2: start / stop / continue

START

What we should begin doing

- Rehearse rollback in a game-day
- Write a golden set before the feature
- Prompt changes as reviewed PRs

STOP

What's slowing us or adding risk

- Editing prompts straight on main
- Disabling the eval gate 'just once'
- Debugging live during an incident

CONTINUE

What's working — keep it

- Blue/green for every deploy
- Telemetry-first debugging
- Small, traceable commits

Fill these live with the cohort — this board becomes the team's working agreement for Week 3.

Technical confidence map

Rate yourself 1–5 on each skill from the last two weeks. Green = ready to lead it; amber = can do with help; red = needs a session.

- Containerise a Python agent
- Container Apps vs AKS decision
- GitHub Actions CI/CD (OIDC)
- Build an eval gate
- Model-card automation
- Telemetry: token / p99 / cost
- Push to ACR + managed identity
- KEDA auto-scaling rules
- Blue/green + traffic weights
- Prompt versioning & registry
- Execute the rollback runbook
- Drift & Cohen's Kappa

Where a skill is red for many, that's a Week-3 clinic. Where you're green, you'll mentor a peer.

Week-3 readiness checklist

Deploys are boring

Every merge to main builds, gates, and blue/green-deploys with no manual steps.

The gate has teeth

A real golden set exists and a regression actually blocks a deploy — you've seen it fail.

Rollback is rehearsed

The 02:00 runbook is pinned, the command is known, and you've run it in a game-day.

You can see quality

Token, p99 and an eval/faithfulness monitor are wired to alerts — not just HTTP metrics.

Releases are documented

A model card is generated per release; prompts are versioned in a registry.

Capstone scoped

You know your agent, its golden set, its scale rules, and its rollback play.

If you can tick all six, you're ready for Week 3. Any gaps become this week's focus.

TECHADEMY

07

WEEK-2 CAPSTONE

Ship it — with the safety rails

An Azure-deployed agentic system

Capstone Scope:

keep the agent small (one clear job - e.g. KYC triage, ticket routing, or doc Q&A) so the effort goes into the rails, not the model.

The grade is on the operational envelope - can you deploy safely, prove quality, see it in production, and undo a bad change in under a minute (not on how clever the agent is)

An Azure-deployed agentic system

Containerised agent

The Python agent Dockerised and running on Azure Container Apps, image in ACR, pulled via managed identity.

CI/CD live with blue/green

GitHub Actions builds, gates, and blue/green-deploys on every merge to main — no manual steps.

Eval gates

A golden set + an eval gate that provably blocks a regression >2% below baseline.

Telemetry

Token counts, p99 latency and an eval/faithfulness monitor flowing to Application Insights.

Rollback play

A pinned 02:00 runbook and a one-command traffic-flip rollback you've rehearsed.

Release docs

Prompts versioned in a registry; a model card auto-generated per release.

Same spine as the BFSI go-live: deploy → gate → observe → roll back. Your domain, your agent — these rails are non-negotiable.

Capstone acceptance criteria

- ✓ docker run works locally; the image is a slim multi-stage build tagged by commit SHA.
- ✓ The app runs on Container Apps with a scale rule (min/max replicas) that you can justify.
- ✓ A push to main triggers GitHub Actions: build → eval gate → blue/green deploy.
- ✓ You can demonstrate the eval gate FAILING a deliberately-bad change and blocking the deploy.
- ✓ Blue/green is live: green validated on its label URL before a 100% traffic switch.
- ✓ Rollback rehearsed: one command returns 100% traffic to the last-good revision in seconds.
- ✓ Telemetry shows per-request tokens, p99 latency and a quality signal in Application Insights.
- ✓ Each release ships a versioned prompt id and an auto-generated model card.

Deploy & container

Azure Container Apps (ACA) Serverless containers on managed Kubernetes; no cluster to run.

AKS Azure Kubernetes Service — full Kubernetes, full control.

KEDA Event-driven autoscaler; powers ACA scale rules (incl. to zero).

Dapr / Envoy Building-block APIs / the proxy handling ingress & traffic split.

Revision An immutable snapshot of a container app version.

Scale-to-zero Drop to zero replicas when idle — zero billing.

Cold start Latency to wake a scaled-to-zero app (~5–30s).

ACR Azure Container Registry — private store for images.

Managed identity Passwordless Azure identity; used with AcrPull to pull images.

Dockerfile The recipe that builds a container image.

Multi-stage build Build deps in one stage, ship only the runtime — smaller image.

Consumption profile ACA pay-per-second serverless tier with scale-to-zero.

CI/CD & blue-green

CI/CD Automated build, test and deploy on every code change.

GitHub Actions GitHub's YAML-defined CI/CD workflows.

OIDC (federated) Short-lived cloud auth from CI — no stored secret.

Blue/green Two revisions; shift traffic from old (blue) to new (green).

Revision label A stable name (blue/green) with its own test URL.

Traffic weight The % of traffic routed to a revision; must sum to 100.

activeRevisionsMode Set to 'multiple' so both revisions run at once.

Canary Gradual traffic shift (e.g. 10% at a time) with monitoring.

Eval gate A CI job that blocks a deploy if quality regresses.

Golden set Versioned reference cases the eval runs against.

Baseline Current production scores the new build is compared to.

github.sha The commit hash used as the immutable image tag.

LLMOps & runbook

LLMOps Ops for LLM apps: versioning, evals, monitoring, rollback.

Prompt versioning Treating each prompt as a reviewed, versioned artifact.

Prompt registry The single source of truth for active prompt versions.

Model card Structured doc: intended use, evals, limits — for auditors.

Rollback Reverting to the last-good state (traffic flip / prompt id).

Feature flag A switch to enable/disable a path without a deploy.

Runbook A pinned, tested set of steps to handle an incident.

SEV (severity) Incident level; SEV-1 = highest, act now.

Incident commander The single owner coordinating an incident.

Post-mortem Blameless review; adds the failure to the golden set.

Game-day A rehearsed failure drill to test the runbook.

MTTR Mean time to restore — a headline reliability SLO.

EU AI Act Regulation requiring high-risk AI documentation.

Faithfulness Whether an answer is grounded in its source / data.

REFERENCE

Sources & further reading

Compare Azure container options (ACA vs AKS vs Functions)	learn.microsoft.com
ACA vs AKS decision framework (2026)	developersvoice.com
Blue-green deployment in Azure Container Apps	learn.microsoft.com
Traffic splitting in Azure Container Apps	learn.microsoft.com
Deploy to Azure Container Apps with GitHub Actions	learn.microsoft.com
Azure/container-apps-deploy-action	github.com
Automated LLM eval — a CI/CD quality gate that runs	galtea.ai
Model Cards (format, metadata, automation)	huggingface.co

TECHADEMY

RECAP

Deploy fast — undo faster

- Container Apps ships the agent live; AKS is the read-ahead you graduate to when constrained.
- Dockerise slim, push to ACR by commit SHA, and scale with KEDA rules (even to zero).
- GitHub Actions + blue/green + an eval gate: bad quality never reaches users.
- LLMOps: version prompts, auto-generate model cards, and keep rollback one command away.
- At 02:00, the rehearsed runbook — not a heroic fix — is what stops a SEV-1.